

Name :- Rucha Nitin Pawar

Branch :- AI&DS

Div :- B

Roll No :- 70

Batch :- B4

Aim:- Data wrangling with pandas: Manipulates and transform data to suit the analysis needs.

1. i) What is data wrangling ?

Data wrangling is the process of transforming and organizing raw data into a clean and structured format suitable for analysis. It involves tasks like cleaning messy data, handling missing values, reshaping datasets, merging data from different sources, and converting formats so the data becomes usable.

ii) Explain the difference between data wrangling and data preprocessing?

The main difference between data wrangling and data preprocessing is their scope and purpose. Data wrangling is a broader process focused on preparing raw, unstructured data into a usable form, often involving gathering and restructuring data from multiple sources. Data preprocessing, on the other hand, is a more specific step usually performed after wrangling, where the cleaned data is further refined for analysis or machine learning, such as normalization, encoding categorical variables, and scaling features. In short, data wrangling makes data usable, while data preprocessing makes it ready for modeling.

2. What is importance of exploratory data analysis (EDA) in pandas?

Exploratory Data Analysis (EDA) in pandas is important because it helps you understand the structure, patterns, and quality of a dataset before performing detailed analysis or building machine learning models. Using pandas functions like `head()`, `describe()`, `info()`, and visual summaries, you can quickly identify missing values, detect outliers, understand distributions, and find relationships between variables. This process ensures better decision-making, helps choose the right analysis techniques, and improves the accuracy and reliability of results.

3. What are the main differences between series, DataFrame, and Panel?

In pandas, the main differences between Series, DataFrame, and Panel are based on their dimensionality and structure:

1. Series – A one-dimensional labeled array that can hold any data type (integers, strings, floats, etc.). Each element has an index label.

Example: `s = pd.Series([10, 20, 30])`

2. DataFrame – A two-dimensional labeled data structure, like a table, with rows and columns. Each column can have a different data type. Example: `df = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})`

3. Panel – A three-dimensional data structure (like a container of DataFrames), used for storing data with three axes: items, major_axis, and minor_axis. Panel is deprecated in recent pandas versions in favor of MultiIndex DataFrames or xarray. Example (legacy): `p = pd.Panel(np.random.rand(2, 4, 5))`

4. Explain the use of: `a.isnull()`, `b.notnull()`, `c.dropna()`, `d.fillna()`?

a. `a.isnull()` – Checks for missing values in a DataFrame or Series and returns a boolean mask, with `True` where values are null (NaN) and `False` otherwise.

b. `b.notnull()` – Opposite of `isnull()`, it returns `True` for non-missing values and `False` for null values.

c. `c.dropna()` – Removes any rows (or columns, if specified) that contain missing values, returning a cleaned DataFrame without NaNs.

d. `d.fillna()` – Replaces missing values with a specified value or method (like forward-fill or backward-fill), allowing you to fill gaps in the data without dropping rows.

These functions are essential for detecting, handling, and cleaning missing data in datasets.

5. What is column transformation and why is it important data analysis?

Column transformation is the process of modifying, creating, or encoding the columns of a dataset to make the data suitable for analysis or machine learning. This can include operations like scaling numeric values, encoding categorical variables, creating new features from existing columns, or applying mathematical transformations.

```
import pandas as pd
```

```

# 1. Create Sample Dataset
data = {
    "name": ["Amit", "Riya", "John", "Neha", None],
    "age": [23, 25, None, 28, 30],
    "city": ["Pune", "Mumbai", "Delhi", "Pune", "Mumbai"],
    "salary": ["25000", "30000", "28000", None, "35000"]
}

df = pd.DataFrame(data)

print("Original Data:\n", df)

# 2. Handling Missing Values
df["name"].fillna("Unknown", inplace=True)
df["age"].fillna(df["age"].mean(), inplace=True)
df["salary"].fillna("30000", inplace=True)

# 3. Renaming Columns
df.rename(columns={
    "name": "Name",
    "age": "Age",
    "city": "City",
    "salary": "Salary"
}, inplace=True)

# 4. Changing Data Types
df["Salary"] = df["Salary"].astype(int)
df["Age"] = df["Age"].astype(int)

# 5. Filtering Data (Age > 25)
filtered_df = df[df["Age"] > 25]

# 6. Adding New Column
df["Salary_in_K"] = df["Salary"] / 1000

# 7. Grouping Data (Average Salary by City)
grouped_df = df.groupby("City")["Salary"].mean()

# 8. Displaying Results
print("\nCleaned and Transformed Data:\n", df)
print("\nFiltered Data (Age > 25):\n", filtered_df)
print("\nAverage Salary by City:\n", grouped_df)

```

Original Data:

	name	age	city	salary
0	Amit	23.0	Pune	25000
1	Riya	25.0	Mumbai	30000
2	John	NaN	Delhi	28000
3	Neha	28.0	Pune	None
4	None	30.0	Mumbai	35000

Cleaned and Transformed Data:

	Name	Age	City	Salary	Salary_in_K
0	Amit	23	Pune	25000	25.0
1	Riya	25	Mumbai	30000	30.0
2	John	26	Delhi	28000	28.0
3	Neha	28	Pune	30000	30.0
4	Unknown	30	Mumbai	35000	35.0

Filtered Data (Age > 25):

	Name	Age	City	Salary
2	John	26	Delhi	28000
3	Neha	28	Pune	30000
4	Unknown	30	Mumbai	35000

Average Salary by City:

City	Salary
Delhi	28000.0
Mumbai	32500.0
Pune	27500.0

Name: Salary, dtype: float64

/tmp/ipykernel_1738/783706620.py:16: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chain. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df

```
df["name"].fillna("Unknown", inplace=True)
/tmp/ipykernel_1738/783706620.py:17: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chain
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are sett

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df

df["age"].fillna(df["age"].mean(), inplace=True)
/tmp/ipykernel_1738/783706620.py:18: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chain
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are sett

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df

df["salary"].fillna("30000", inplace=True)
```

Conclusion:

Data wrangling with pandas is a powerful and efficient way to clean, reshape, and prepare data for analysis. By handling missing values, filtering irrelevant information, transforming data types, and organizing datasets, we can significantly improve data quality and usability. This process ensures that the final dataset is accurate, consistent, and ready for meaningful analysis or visualization. Ultimately, effective data wrangling leads to better insights and more reliable decision-making.